



Objektové programování

📄 State	Done
🕒 Edited	@January 10, 2026 7:10 PM
🕒 Created	@November 12, 2025 10:33 AM
🔍 To Review	☐
★ Favorite	☐
🗄 Archive	☐
📌 Topics	 <u>Odborná maturita teoretické otázky (SWI)</u>

Objektové programování (jmenné prostory, dědičnost, zapouzdření, vlastnosti, statika, abstrakce, interface, polymorfismus, přetížení, generika)

Definice

Objektově-orientované programování (OOP) je programovací paradigma, kde data jsou reprezentována tzv. objekty

Objekty

- zapouzdřují stav (data)
 - veřejná data = data zpřístupnitelná zvenčí objektu
 - privátní data = data se kterými objekt vnitřně pracuje, ale nezpřístupňuje je na venek
- ukládají data v atributech nebo vlastnostech
- poskytují rozhraní pro práci s vnitřním stavem

Dědičnost

- Mechanismus, který umožňuje novým třídám (potomkům) přebírat atributy a metody existujících tříd (rodičů). Potomek tak rozšiřuje nebo modifikuje chování nadřazené třídy.
- objekty jsou strukturovány stromovým způsobem
- obvyklá implementace pomocí tříd

Zapouzdření (Encapsulation)

- skrytí vnitřního stavu objektu
- data jsou přístupná pouze přes metody { get; set; }
- ochrana integrity dat

Vlastnosti (Properties)

- Moderní způsob realizace zapouzdření (zejména v C#). Kombinují pole s metodami pro čtení a zápis do jednoho prvku.

Statika (Static)

= Členy třídy (pole, vlastnosti, metody), které jsou sdílené všemi instancemi

- **Statické pole/vlastnost:** Existuje v paměti pouze **jednou** (sdílená hodnota)
- **Statická metoda:** Lze ji volat přímo přes název třídy, aniž by se vytvářela instance. Nemůže přistupovat k nestatickým prvkům.
- Statická třída: nelze vytvořit instanci. Pouze statické vlastnosti.

Abstrakce

- zaměření na podstatné vlastnosti a skrytí nepodstatných detailů (např. u studenta váhu nebo barvu vlasů, pokud řešíme registraci)
- hledání společných vlastností společného typu objektů
- **Abstraktní třída**
 - nelze z ní vytvořit instanci
 - slouží jako nadřazený vzor

Interface (rozhraní)

= Sada metod a vlastností, které musí třída implementovat, pokud se k tomuto rozhraní přihlásí

- Vlastnosti
 - klíčové slovo `interface`
 - Podle konvence začíná názvem na velké "I" (např. `IMobil`, `IDatabase`).
 - Ve většině případů neobsahuje implementaci, ale pouze hlavičky metod
 - Třída může implementovat **více rozhraní najednou**
- Použití
 - Sjednocení
 - Náhrada za vícenásobnou dědičnost
 - Flexibilita

```
interface INabijeci {  
    void Nabit(); // Jen předpis, žádné tělo metody  
}  
  
class Telefon : INabijeci {  
    public void Nabit() => Console.WriteLine("Telefon se nabíjí přes USB-C.");  
}  
  
class Auto : INabijeci {  
    public void Nabit() => Console.WriteLine("Auto se nabíjí v nabíjecí stanici.");  
}
```

Polymorfismus

= Jedno rozhraní, mnoho implementací

- každá operace může mít více implementací na základě typu objektu, se kterým se operace provádí
- **Statický polymorfismus (přetížení)**
 - Děje se již při **překladu programu**
 - Přetěžování metod
 - Více metod se stejným názvem v jedné třídě, které se liší počtem nebo typem parametrů

```
class Kalkulacka {
    // Stejný název, různé parametry
    public int Secti(int a, int b) => a + b;
    public double Secti(double a, double b) => a + b;
}

Barva modra = new Barva("Modrá");
Barva zluta = new Barva("Žlutá");

// Operátor + vytvoří novou instanci s názvem "Modrá-Žlutá"
Barva zelena = modra + zluta;

Console.WriteLine(zelena.Nazev); // Modrá-Žlutá
```

- Přetěžování operátorů
 - Možnost definovat, co má dělat např. operátor **+** při sčítání dvou objektů třídy *Zlomek*.

```
class Barva {
    public string Nazev { get; set; }

    public Barva(string nazev) => Nazev = nazev;

    // Přetížení operátoru +
```

```

        public static Barva operator +(Barva b1, Barva b2)
        {
            return new Barva(b1.Nazev + "-" + b2.Nazev);
        }
    }

```

- **Dynamický polymorfismus**

- Děje se až **za běhu programu**
- Program pracuje s objektem skrze jeho předka (obecný typ), ale vykoná se metoda konkrétního potomka.
- Prvky:
 - **Virtuální metoda (`virtual`)**: Metoda v rodičovské třídě, u které dáváme svolení k tomu, aby byla v potomcích přepsána.
 - **Překrytí metody (`override`)**: Metoda v dceřiné třídě, která definuje novou (vlastní) logiku.

```

// Rodičovská třída
class Zvire {
    public virtual void VydajZvuk() => Console.WriteLine
("Zvíře vydává zvuk");
}

// Dceřiné třídy
class Pes : Zvire {
    public override void VydajZvuk() => Console.WriteLine
("Haf!");
}

class Kocka : Zvire {
    public override void VydajZvuk() => Console.WriteLine
("Mňau!");
}

```

```
// Praktické použití v programu
Zvire mojeZvire = new Pes();
mojeZvire.VydajZvuk(); // Vypíše "Haf!", i když je proměnná typu Zvire
```

Generika (Generické typy)

= Umožňují psát třídy, rozhraní nebo metody, které pracují s různými datovými typy, aniž by ztrácely informaci o tom, o jaký typ jde.

- symbolem pro typ je nejčastěji písmeno `<T>`
- Výhody
 - Typová bezpečnost
 - Znovupoužitelnost
 - Výkon

Kompozice

- alternativa dědičnosti
- třída nepřebírá vlastnosti jiné třídy, ale obsahuje ji jako atribut
 - dědičnost = IS-A
 - kompozice = HAS-A

příklad dědičnosti:

```
class Animal
{
    public string Name { get; }

    public Animal(string name)
    {
        Name = name;
    }

    public void Eat()
```

```

        {
            Console.WriteLine($"{Name} is eating.");
        }
    }

    class Dog : Animal
    {
        public Dog(string name) : base(name)
        {
        }

        public void Bark()
        {
            Console.WriteLine($"{Name} says: Woof!");
        }
    }

    // použití
    var dog = new Dog("Rex");
    dog.Eat();    // zděděná metoda
    dog.Bark();   // vlastní chování

```

příklad kompozice:

```

class Engine
{
    public int HorsePower { get; }

    public Engine(int horsePower)
    {
        HorsePower = horsePower;
    }

    public void Start()
    {
        Console.WriteLine($"Engine with {HorsePower} HP start

```

```

ed.");
    }
}

class Car
{
    public string Model { get; }
    private Engine engine;

    public Car(string model, Engine engine)
    {
        Model = model;
        this.engine = engine;
    }

    public void Start()
    {
        Console.WriteLine($"{Model} is starting...");
        engine.Start(); // delegace
    }
}

// použití
var engine = new Engine(150);
var car = new Car("Škoda Octavia", engine);

car.Start();

```